

C语言与程序设计

The C Programming Language

第1章 概论

王多强

QQ: 1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院



群名称: C语言学习群-2020

群 号: 1139598770

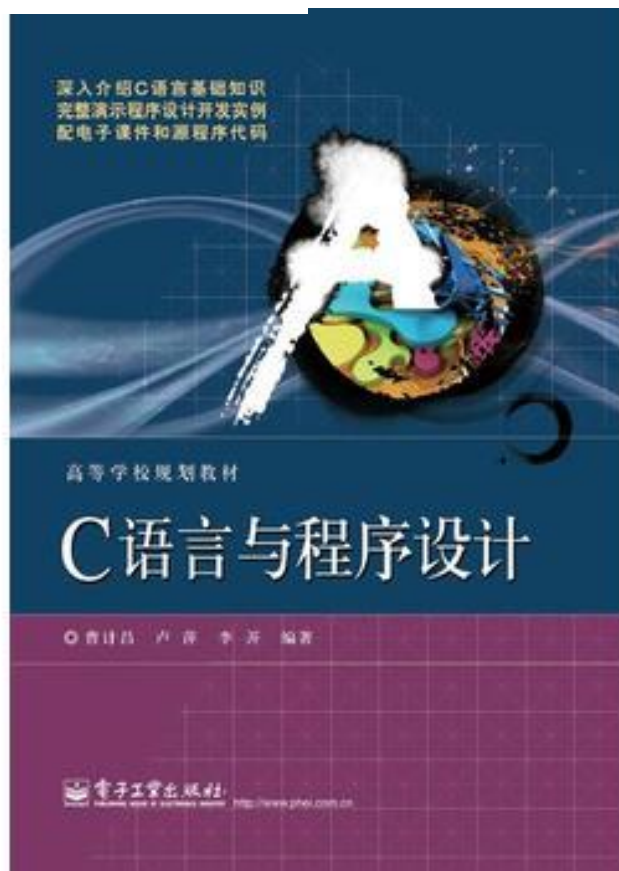
■ 请加群：



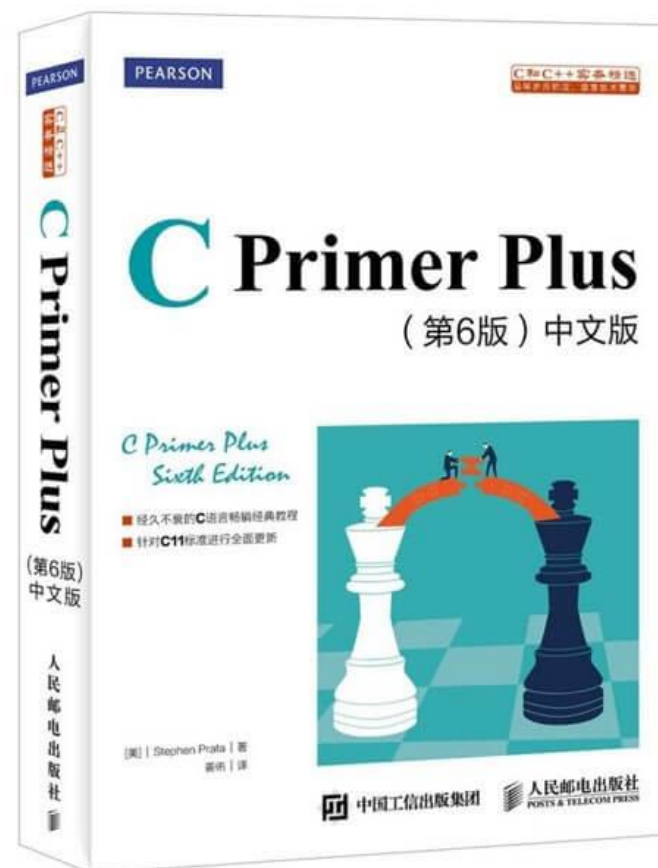
群名称：C语言学习群-2020

群 号：1139598770

■ 教材:



■ 参考书:



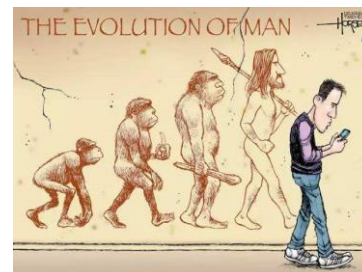
本章主要内容

- 
- 1.1 计算机发展的历史
 - 1.2 程序设计语言与程序设计
 - 1.3 C语言的产生、发展与语言特征
 - 1.4 理解计算机
 - 1.5 数和字符的编码表示
 - 1.6 算法及其表示
 - 1.7 学习C语言与程序设计的方法
 - 1.8 手把手：第一个程序例子

1.1 计算机发展的历史

电子数字计算机是20世纪人类最伟大的发明之一。数字计算机从诞生到广泛应用，今天已深刻改变了我们的世界，如今计算机成为我们工作、学习和生活中不可或缺的一部分。

电子数字计算机已在世界范围内形成了一种新的文化，构造了一种崭新的文明。



1. 人类计算的梦想

我们的祖先怎么计数：

远古时代：掰手指，垒石块

新石器时代：结绳



近 古 代：

➤ 人类最早有实物作证的计算工具诞生在中国：算筹

“运筹帷幄之中，决胜千里之外”

➤ 中国古代的手工“计算机”：算盘



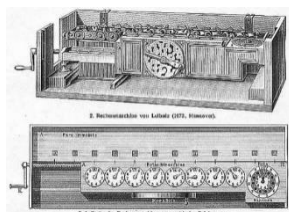
Digit这个单词是什么意思？

- ◆ 既表示“手指”
- ◆ 又表示“整数数字”

2. 计算进入机械时代

人类对于**计算机器**的梦想一直没有间断

- ◆ 17世纪，法国科学家布莱斯·帕斯卡(Blaise Pascal)发明了**加法器**



第一次确立了计算机器的概念

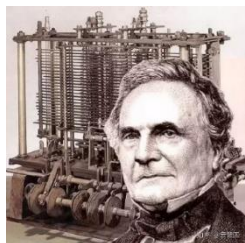
- ◆ 1674年，德国数学家莱布尼茨(Gottfried Wilhelm Leibniz)发明了**乘法器**。



莱布尼兹, G. W.

1700年左右，莱布尼茨受中国“易图”（八卦）的启发，最终悟出了**二进制数**的真谛。

- ◆ 18世纪末，英国剑桥大学教授查尔斯·巴贝奇 (Charles Babbage) 发明了**差分机**



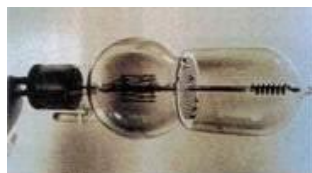
它能够自主完成三组十万以内的加法，而不需要任何人力计算

巴贝奇被视为现在计算机的奠基人

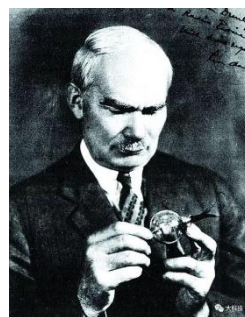
3. 电子数字计算机的诞生

1) 电子文明的曙光——电子二极管、三极管

- ◆ 1904年，英国电机工程师约翰·弗莱明发明了真空二极管



- ◆ 1906年，美国人李·德·福雷斯特发明了真空三极管

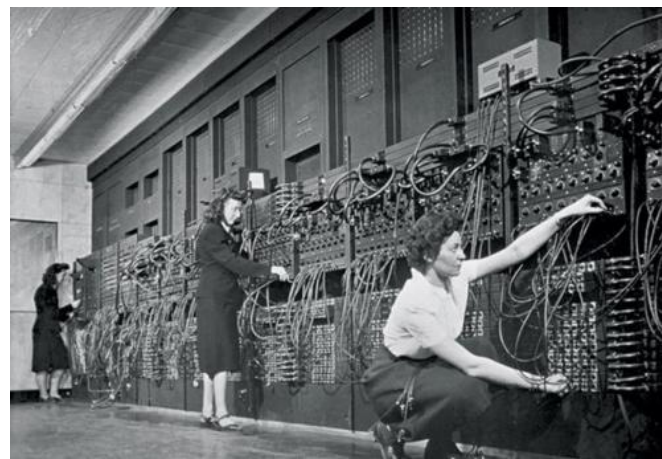


电子管的诞生为通讯、广播、电视等技术的发展铺平了道路。
计算机的历史也由此跨进电子时代的新纪元。

2) 第一台电子数字计算机的诞生——ENIAC

1946年2月14日，世界第一台电子计算机在美国宾夕法尼亚大学诞生，发明人是美国人**莫希利**和 (他的学生) **埃克特**，用于**炮弹弹道轨迹计算**。

ENIAC: Electronic Numerical Integrator And Computer, 埃尼阿克
电子数值积分和计算机 (1943-1946)



- ◆ ENIAC总共安装了**17468**只电子管，7200个二极管，70000多电阻器，10000多只电容器和6000只继电器。占地面积**170**平米，总重量30吨，耗电174千瓦。
- ◆ ENIAC的运算速度：**每秒5000次加法**，可在0.003秒内完成两个10位数乘法。

ENIAC有多快呢？

以圆周率(π)的计算为例，

- ◆ 中国古代科学家祖冲之利用算筹，耗费15年心血，才把圆周率计算到小数点后7位数。
- ◆ 一千多年后，英国人香克斯以毕生精力计算圆周率，计算到小数点后707位。
- ◆ 而ENIAC仅用了40秒就达到了这个记录，还发现了香克斯的数据中第528位是错误的。
- ◆ 解决复杂的弹道计算问题，由20分钟缩短到30秒，速度提高40倍。

ENIAC标志着电子计算机的创世，人类社会从此迈入电子计算机新时代。

现在历史公认：真正的第一台计算机是阿塔纳索夫-贝瑞计算机，简称**ABC**计算机。诞生在1937年，不可编程，仅用于求解线性方程组。

◆ 但ENIAC并不是完整意义下的现代计算机。

- 采用十进制而非二进制
- 非冯·诺依曼体系结构

美籍匈牙利数学家冯·诺依曼于1946年提出**"存储程序"原理**：

- 把程序本身当作数据来对待，程序和该程序处理的数据用同样的方式首先**储存在机器里**，机器启动后应该**自动地按照程序顺序执行**；
- **计算机的数制采用二进制。**

人们把冯·诺依曼的这个理论称为**冯·诺依曼体系结构**。

◆ 现代意义下的第一台通用计算机：EDVAC

- 采用二进制
- 采用冯·诺依曼体系结构

EDVAC： Electronic Discrete variable Automatic Computer
离散变量自动电子计算机。

冯·诺依曼于1945年6月发表了著名的101页报告：“关于EDVAC的报告草案”（en:First Draft of a Report on the EDVAC），明确奠定了新机器由五个部分组成，包括：**运算器、逻辑控制装置(控制器)、存储器、输入设备和输出设备。**

报告提出的体系结构一直延续至今，即**冯·诺伊曼结构**。

1946年，普林斯顿高等研究院（the Institute for Advance Study at Princeton, IAS）开始设计“**存储程序**”式计算机，被称为**IAS计算机**，但这台计算1951年才完成，使得它并不是世界上第一台存储程序计算机。世界上第一台存储程序计算机是1949年由英国剑桥大学完成的**EDSAC**。¹²

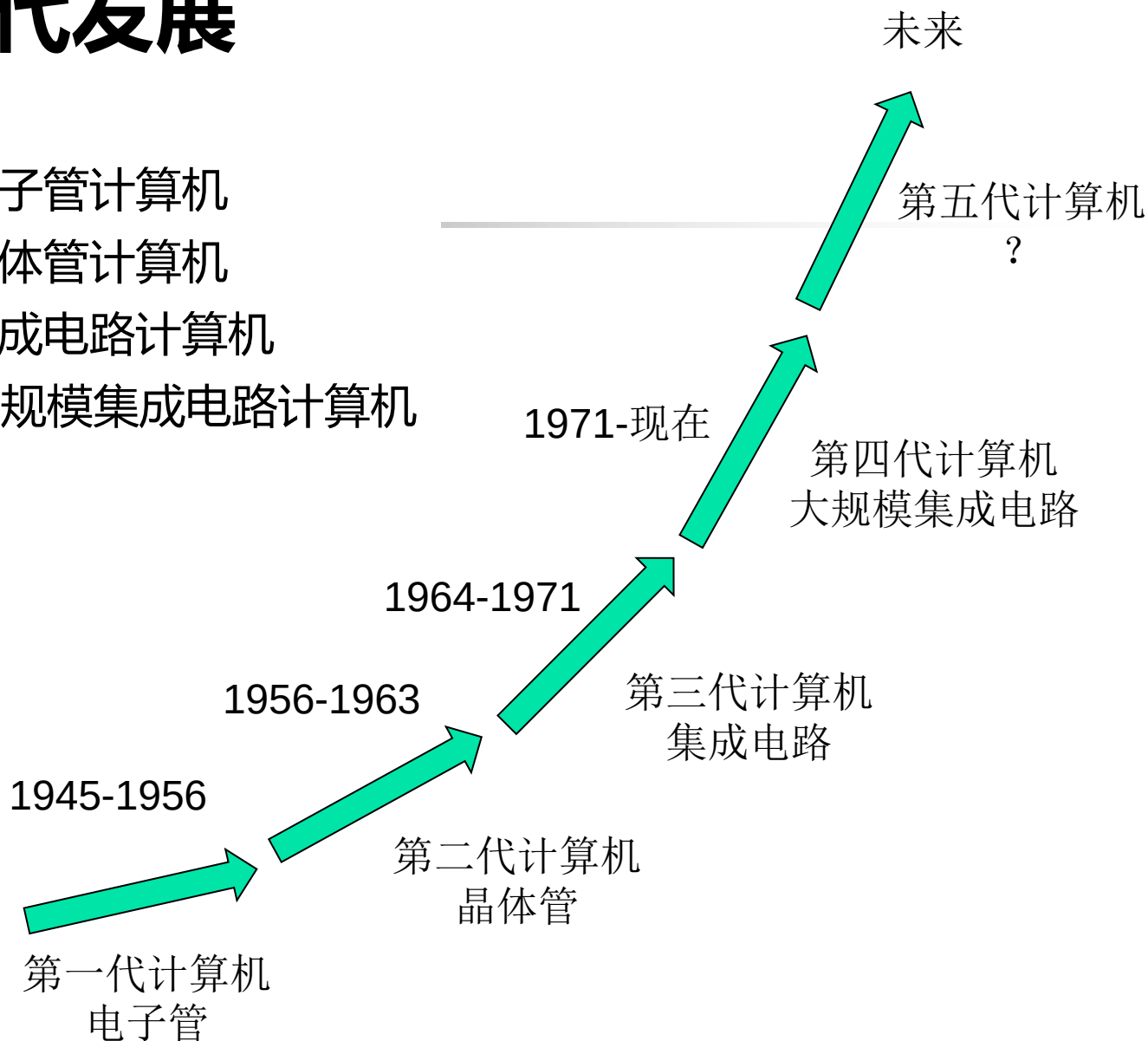
冯·诺伊曼结构中各基本部件的功能是：

- **存储器**：存放数据和指令，两者形式上没有区别，但计算机应能区分数据还是指令；
- **控制器**：能够自动取出指令并控制指令按顺序依次执行；
- **运算器**：进行基本的算术运算、逻辑运算和其它附加运算；
- 操作人员可以通过**输入设备**、**输出设备**和主机进行通信。

计算机的历代发展

- 第一代计算机：电子管计算机
- 第二代计算机：晶体管计算机
- 第三代计算机：集成电路计算机
- 第四代计算机：大规模集成电路计算机
- ...
- 第N代计算机：？

体积越来越小
速度越来越快
功能越来越强大



3) 晶体管时代——第二代计算机：晶体管计算机

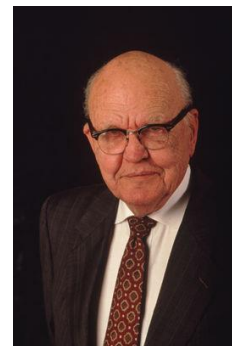
1947年12月23日，美国科学家**巴丁**、**布莱顿**、**肖克莱**发明了科技史上具有划时代意义的成果——**晶体管**，推动了全球范围内的**半导体电子工业**的发展，集成电路、大规模集成电路应运而生，电子计算机的发展也进入了快速发展的道路。

- ◆ 1954年美国电报电话公司（AT&T）贝尔实验室研制成功第一台半导体计算机TRADIC；
- ◆ 1960年IBM全面推出晶体管化的7000系列电脑，称为第二代电子计算机的典型代表



4) 集成电路时代——第三代计算机：集成电路计算机

1958年9月12日，美国工程师**杰克·基尔比**成功地把20多个电子器件集成在一块面积不超过4平方毫米的半导体材料上，这一天，被视为**集成电路的誕生日**（42年后，2000年，基尔比因集成电路的发明被授予**诺贝尔物理学奖**）。



当时的**仙童半导体公司**迅速做出反应，开始生产制造计算电路。

- 1964年，仙童公司的戈登·摩尔（G.Moore）博士发表了一个奇特的理论：**集成电路上能被集成的晶体管数目，将会以每18个月翻一番的速度稳定增长，并在今后数十年内保持着这种势头**。摩尔的预言在后来集成电路的发展史上得以证实，被人誉为 **“摩尔定律”**。



5) 大规模集成电路时代

——第四代计算机：大规模/超大规模集成电路计算机

小规模集成电路 (SSI) : 10-100个元件

中规模集成电路 (MSI) : 100-1000个元件

大规模集成电路 (LSI) : 1000个以上的元件

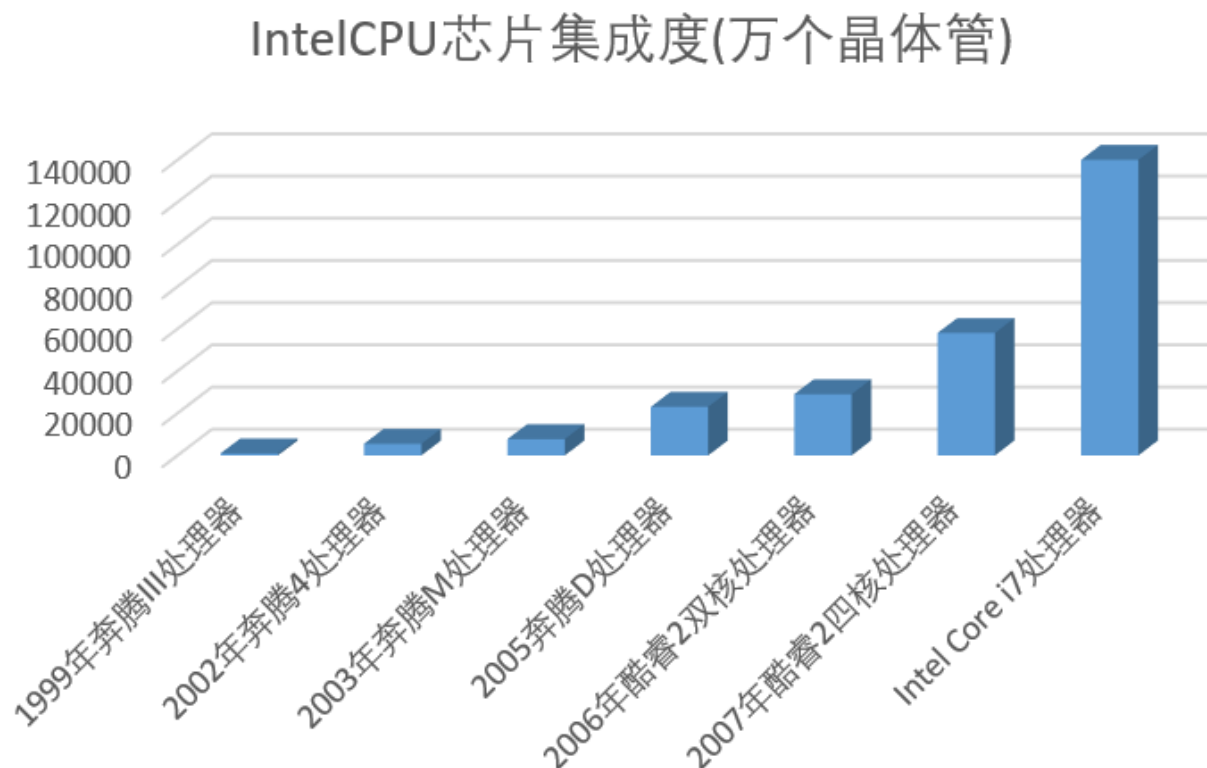
超大规模集成电路 (VLSI) : 10万+个元件

特大规模集成电路 (ULSI) : 1000万+个元件

巨大规模集成电路 (GSI) : 1亿+个元件



Intel CPU 芯片的集成度



1999年奔腾III：950万
2002年奔腾4：5500万
2003年奔腾M：7700万
2005年奔腾D：2.3亿
2006年酷睿2双核：2.9亿
2007年酷睿2四核：5.8亿
Intel Core i7处理器：14亿

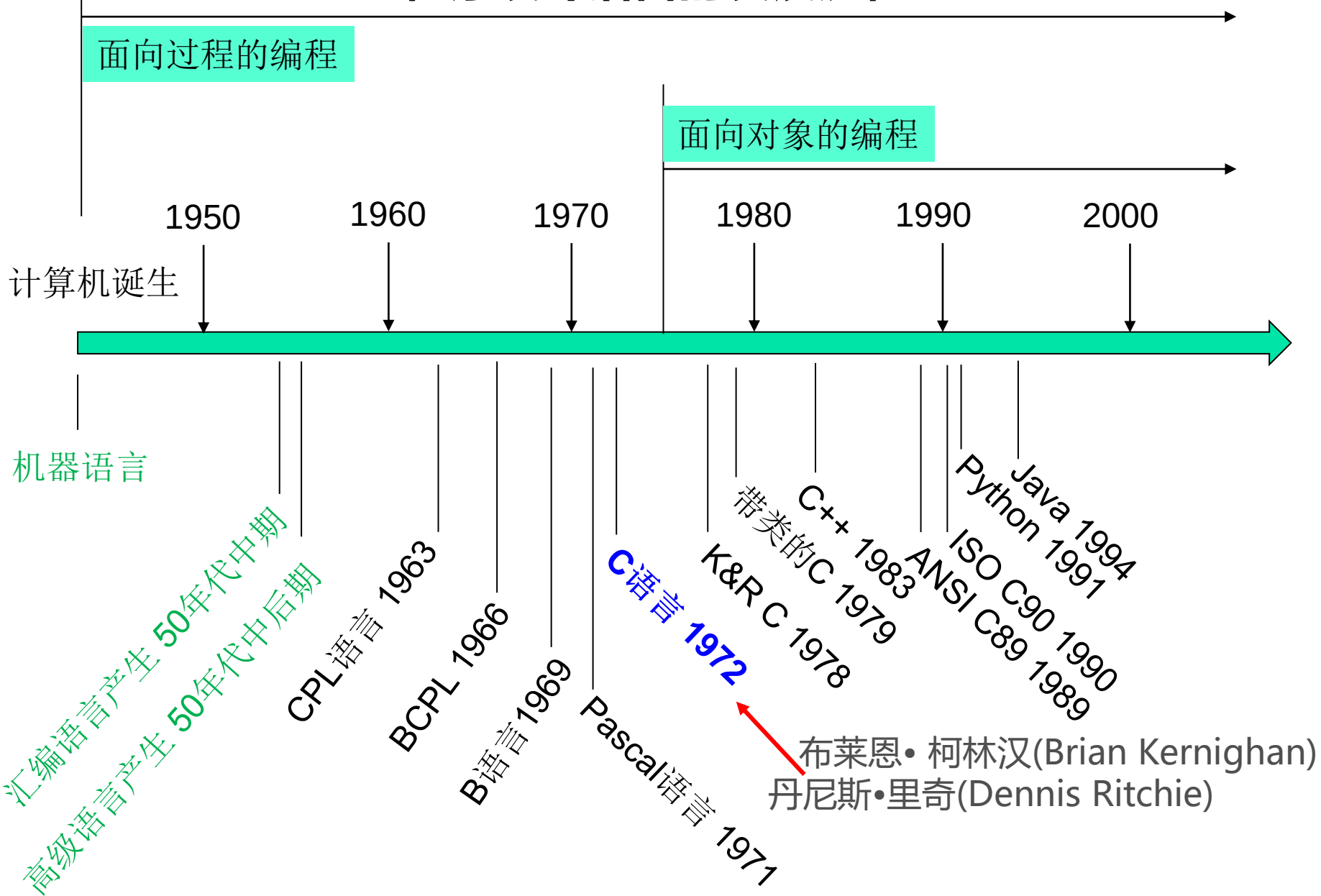
程序设计语言是以具有特定语义的符号为基本构成单位、以语法为程序构成规律、专门用于定义、组织计算机指令以完成各种各样**计算任务**的语言体系。

对比：“自然语言”之于“人”

- 特定语义的符号：字、词
- 以语法为程序构成规律：组词、造句
- 定义、组织、完成各种各样的计算任务：写文章

编程序就是用程序设计语言写一篇“计算机能读懂的文章”

程序设计语言的发展历程



FORTTRAN语言是世界上第一个被正式推广使用的高级语言，1956年开始正式使用

计算机语言三大类型：机器语言、汇编语言、高级语言

1) **机器语言**：是用二进制代码表示的、计算机能直接识别并执行的机器指令的集合。

最早的计算机程序都采用机器语言来编写。

机器指令是长度有限的用特定格式表示的0-1数码序列

如： **1011** **1000** **0011 0100 0001 0010**

操作码

w 目的

源

MOV AX,1234H

机器指令的基本组成： **操作码**：指令的功能，即做什么；

源寄存器/操作数：操作对象；

目的寄存器/地址：操作结果放在那里；

机器语言的特点：

- 机器指令是用0和1组成的一串代码；
- 机器语言程序直接用二进制代码指令表示
- 不用翻译，计算机可以直接执行。

机器语言的缺点：

- **繁琐、难以记忆、难以编写**：适用于非常专业的人士
- **没有数据类型的概念**：任何数据都被表示成“位串”，
不适合面向应用的编程

2) 汇编语言：解决机器语言难以记忆和书写的问题

助记符：帮助记忆的符号，用名称（单词缩写或命名字符串）
表示指令、寄存器等

汇编语言用**助记符**表示指令和指令中的源、目寄存器等。

如：
1011 1000 0011 0100 0001 0010
↓ ↓ ↓
MOV AX, 1234H

优点：使用助记符，**方便记忆**，提高了编程效率

（助记符一般取自指令功能英语单词的缩写）

缺陷：与机器语言一一对应，可视为是“**符号化了的机器语言**”，本质上还是低级语言，用汇编语言编写程序依然很繁琐。

用汇编语言编写的程序称为**汇编语言程序**。但汇编语言程序不能直接执行，需要经过“**汇编程序**”“**汇编**”，翻译成机器语言程序才能为被计算机执行。

——注：**只有机器语言程序才能被计算机直接执行。**

- 机器语言和汇编语言都是低级语言
- 面向机器而不是面向人的
- 没有数据类型的概念
- 对客观世界的问题描述能力差

3) **高级语言**：让计算机语言方便“人”使用

高级语言是用接近于**数学和自然语言**的方式表示计算机指令的程序设计语言。

特点：

- 机器语言和汇编语言是面向机器的，**高级语言是面向用户、面向应用的**；
- 方便人编写程序；
- **引入了数据类型的概念**；

高级语言程序也不能被计算机直接执行，要经过翻译（编译或解释），转换成机器语言程序才能为计算机所执行。

2. 程序与程序设计

- 1) **程序**：就是用程序设计语言表示的计算机解题算法或解题任务，是计算机指令的集合。
- 2) **程序设计**：就是将解题任务转变成程序的过程。
 - ◆ 程序是计算机要执行的**指令序列**；
 - ◆ 程序设计是计划、安排计算机必须遵循的操作步骤及顺序的过程。
 - ◆ **本课程将学习使用一种程序设计语言——C语言进行程序设计的技术。**

1.3 C语言的产生、发展与语言特征



作业：

1. 就C语言的产生发展、标准化历程、语言特征等，写一篇小研究报告，字数 ≥ 500 。
2. 回答问题：什么是语言类型的强弱，为什么说C语言是弱类型的？

1.3.1 C语言的产生与发展

1946年，第1台通用电子计算机 ENIAC 诞生

FORTAN (1956年)

ALGOL 60 (1960年)

CPL (1963年)

BCPL (1967年)

B (1969年-1970年)

C (1971年-1973年)

传统C或K&R C (1978)

ANSI C (1989, 美国国家标准, 俗称C89)

标准 C (ISO/IEC 9899:1990, 俗称C90)

标准 C (ISO/IEC 9899:1999, 俗称C99)

最新标准 C (ISO/IEC 9899:2011, 俗称C11)

图1.1 C语言的继承、产生与发展历程



Martin Richards

1967年，剑桥大学，BCPL编程语言的发明者



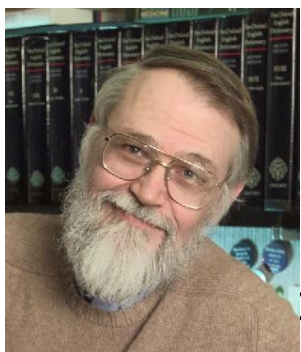
Dennis M. Ritchie

在B语言的基础上设计并实现了C语言并设计了UNIX操作系统。
被称为**C语言之父**、UNIX之父。1983年获得了图灵奖。



Ken Thompson

1973年与Dennis M. Ritchie合作用C语言重写了UNIX操作系统内核。



Brian W. Kernighan

1978年与Dennis M. Ritchie合著了巨著《C Programming Language》，是世界上第一本被广泛认可的C语言教程，该书介绍的C语言被称为**K&R C**（或**传统C**）

1.3.2 C语言标准化

- 以1978年**K&R C**为代表的C语言被称为传统的C语言
- 1989年底公布美国第一个C语言的国家标准ANSI 89，简称**C89**
- 1990年，国际标准化组织ISO将其接受为C语言的国际标准，称为ISO/IEC 9899-1990。它是C语言的第一个国际标准，也称为标准C，简称**C90**。
- ISO/IEC在1995年公布了一个新的C语言标准草案，称为**C95**，供讨论和征求意见。接着ISO/IEC在1998年又公布新标准的草案WG14/N843和WG14/N897，进一步就C语言标准的完善征求意见。
- ISO/IEC于1999年12月公布了C语言国际标准ISO/IEC 9899:1999 (E)。它是C语言国际标准9899的第二版，俗称**C99**。
- ISO/IEC于2011年12月公布了C语言国际标准ISO/IEC 9899:2011。它是C语言国际标准9899的第三版，俗称**C11**。

1.3.3 C语言的特征

- 1) **语言简洁紧凑**：`while(*s++=*t++);`
- 2) **目标代码质量高**：执行效率最高的高级语言
- 3) **语言表达能力强**：数据类型丰富，操作符多、语句集简单且功能强大
- 4) **流程控制结构化**：结构化程序设计
- 5) **弱类型**：使用方便
- 6) **“中级语言”特性**：具备对计算机硬件系统的操纵和控制能力
- 7) **书写自由、使用灵活**
- 8) **可移植性好**：对硬件依赖低，源代码可以移植到不同的计算机上执行

1.4 理解计算机

1.计算机系统的组成

你看到的计算机长什么样？



台式机



笔记本电脑

■ 一般计算机系统由主机和外部设备两大部分组成

主机的关键构成部件：

主机：



台式机主机箱的内部



主板



CPU



内存条



硬盘



显卡



电源



CPU风扇/散热器

外部设备:

输入设备: 采集外部世界的信息, 转换成计算机可处理的数据并输入计算机



键盘



鼠标



摄像头



扫描仪

输出设备: 将计算机内部的数据, 以特定的形式展示出来, 为外部所使用



平板显示器
(液晶、LED)



CRT显示器



激光打印机



针式打印机



绘图仪

2. 冯·诺依曼体系结构

冯·诺依曼的“关于EDVAC的报告草案”给出了现代计算机的体系结构，由五个部分组成：**运算器**、**逻辑控制装置**、**存储器**、**输入设备**和**输出设备**。这一体系结构被称为**冯·诺伊曼结构**。

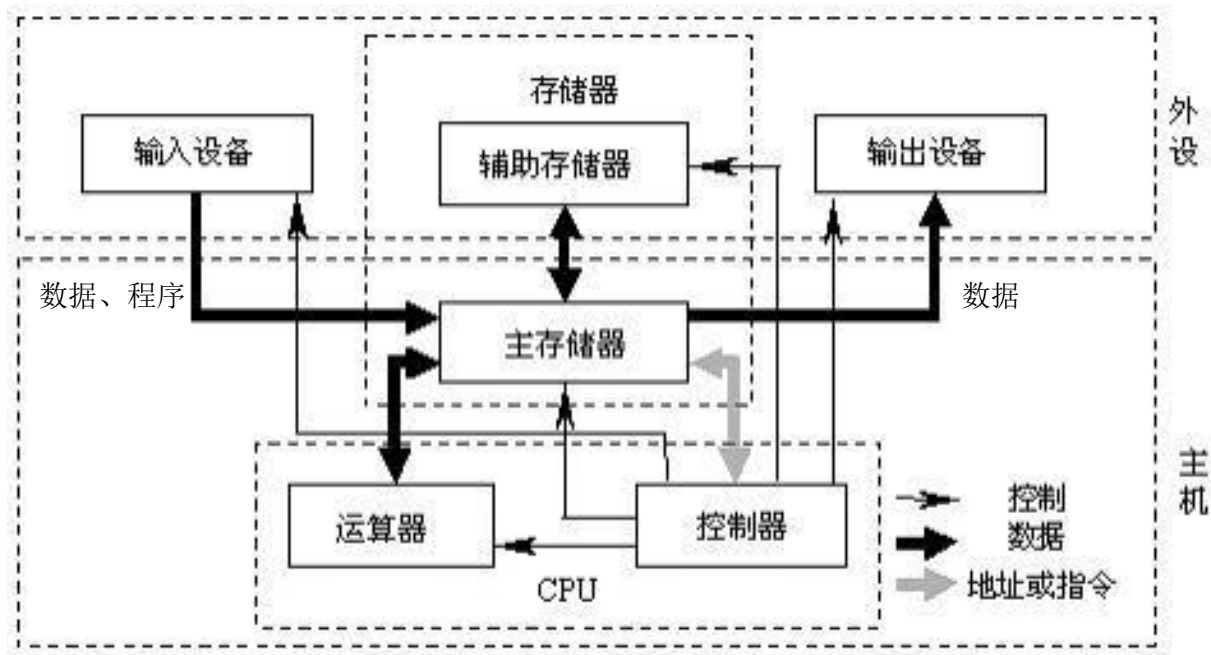


图 1-2 计算机的组成框图

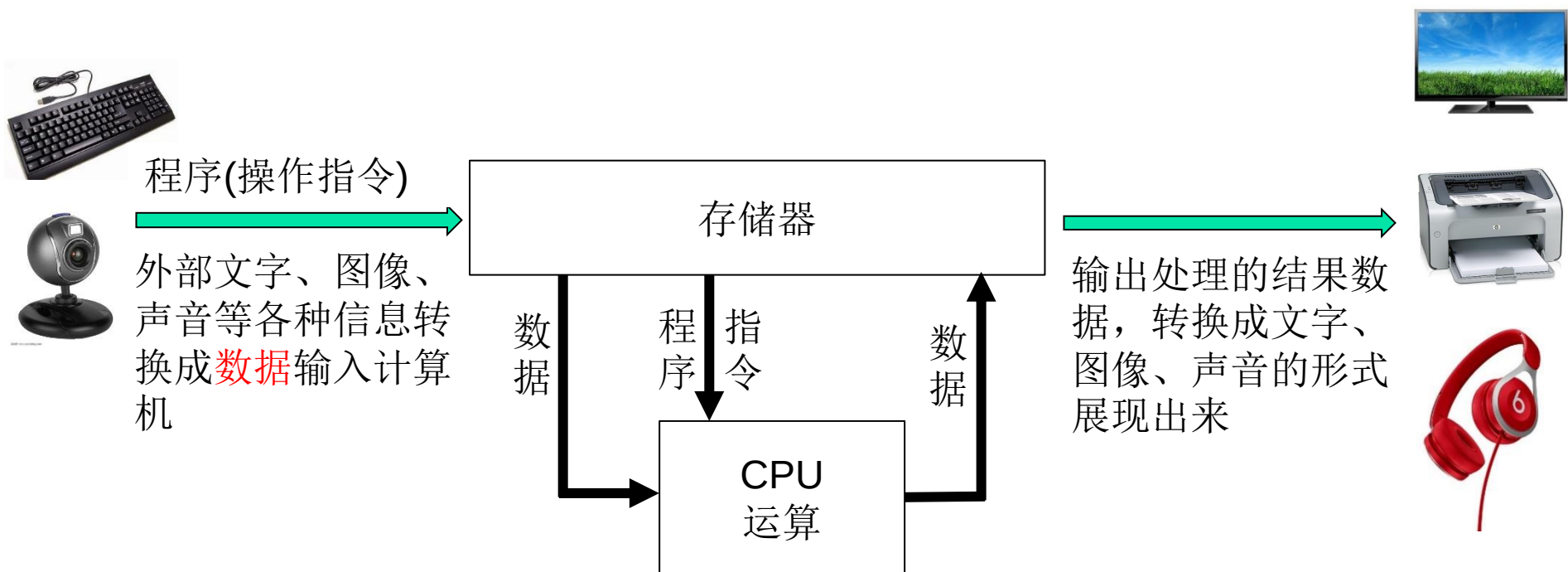
CPU：中央处理器

(Central Processing Unit)

执行程序，对数据进行运算

Memory：存储器

存储数据和程序



运算器：执行运算

运算：算术运算（加减乘除）、逻辑运算（与或非）

算术逻辑单元(ALU, arithmetic and logic unit)

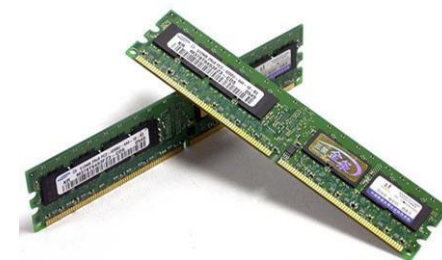
3.存储

- 存储器分**内存**和**外存**

- **外存**：硬盘（包括SSD）、U盘、光盘等；
可以**长期存储数据和程序**，即使关机也不会丢失；
存放在里面的数据和程序以**文件**的形式存在。



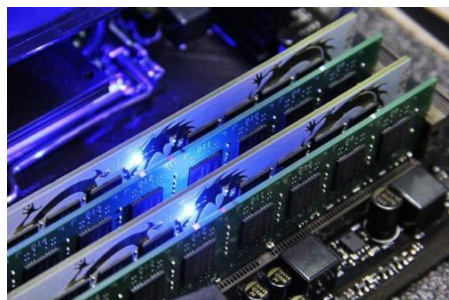
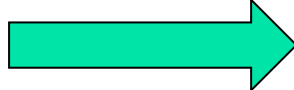
- **内存**：插在主板上的内存条；
只用于机器运行期间**暂时存放数据和程序**，关机后丢失；
在OS的管理下，数据和程序被存放在内存的线性空间里。



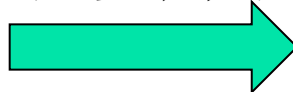
4. 怎么理解内存空间



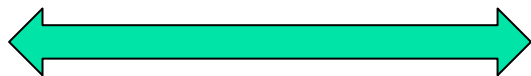
一个内存条相当
于一个书架



多条内存条相
当于多个书架



内存单元



书架上的一个格子

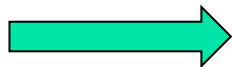
内存空间是所有内存单元的集合 ↔ 所有书架上格子的集合

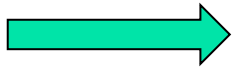
怎么理解内存空间



编号：从第一个书架开始对所有格子进行编号

- ◆ 第一个书架的第一个格子编号为0，后续格子的编号顺次递增，依次为1, 2, ...。
- ◆ 第一个书架的格子全部编完后，第二、三...个书架的格子接着编，编号都依次递增。
- ◆ 若一个书架有N个格子，共有M个书架。则所有格子编号的范围是 $0 \sim M * N - 1$ 。在这个范围内，**每个格子都有一个唯一的编号。**

把格子的编号称为**格子地址**  **内存单元地址**

还可以给每个格子起名字，称为**格子名**  **变量名**



- ◆ 假设每个格子恰好放8本书（假设书都一样）
- ◆ 搬运的时候，为了方便，是一个格子一个格子搬运(注意：不是一本书一本书的搬)

- ◆ 一个格子对应一个内存单元，内存单元称为字节
- ◆ 书架以格子为单位搬运书籍，内存以字节为单位访问。
- ◆ 一个格子可以放8本书，一个内存单元可以存8个位（bit，每个位可以表示一位二进制数值0或1）
- ◆ 书架的格子有格子编号，内存单元有内存地址

内存编址

内存以字节为单位。假定内存空间中有4G个字节，则这些字节的编址范围为 $0 \sim 2^{32}-1$ ，0对应最低地址， $2^{32}-1$ 对应最高地址。

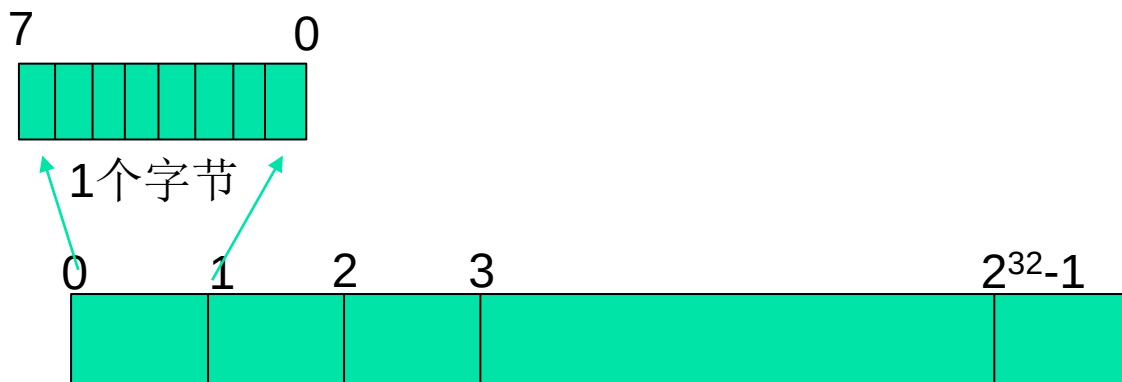


内存编址示意图

- 一般用**B**表示字节 (Byte; 相应地以**b**表示位, bit)
- 内存容量单位: **K**: 2^{10} , 1KB=1024B,
M: 1024K, 1MB=1024KB,
G: 1024M, 1GB=1024MB,
T: 1024G, 1TB=1024GB, **1024T ?**

字节 (Byte) 是计算机最小的存取单位，但计算机最小的信息单位是**位 (bit)**。

一个字节由8个bit组成，每个位可表示二进制的0或1。



内存示意图

注意：位没有独立编址，不能单独直接访问，**要借助特殊的位运算才能读取或修改位的内容。**

1.5 数和字符的编码表示

1.5.1 进位计数制

- ◆ **数目**是表示事物的个数，表示数目的符号称为**数字**，简称为“**数**”。
- ◆ 同样大小的数在不同的**进位制**下有不同的表示形式，形成“**数制**”的概念。
 - **数制**是关于计数的法则
 - 根据进位**基数**的不同，计算机领域常用的进位计数制有**十进制**、**二进制**、**八进制**、**十六进制**等。

1. 二进制数

二进制数的特点：

- (1) 只有0和1两个不同的数字符号（也称为“**数码**”）。
- (2) 运算规则：**逢2进位1**。

二进制数是由**0**和**1**组成的数字串，通常表示为：

如： **$(10110110)_2$**

- 一位二进制数可以表示十进制的0和1：0、1
- 两位二进制数可以表示十进制的0~3：00、01、10、11
- 三位二进制数可以表示十进制的0~7：000、001、010...、111
- n位二进制数可以表示十进制的0~ 2^n-1 ：00...0、...、11...1

2) 二进制数的运算

■ 1位二进制加法规则有四条：

① $0+0=0$

② $0+1=1+0=1$

③ $1+1=0$ 向高位进位1

④ $1+1+1=1$ 向高位进位1

■ 1位二进制减法规则也有四条：

① $0-0=0$

② $1-1=0$

③ $1-0=1$

④ $0-1=1$ 向高位借位1

多位二进制数的加、减：

例1.2 二进制加法：求 $(10110111)_2 + (01011011)_2$ 的和。

$$\begin{array}{r} 10110111 \\ + 01011011 \\ \hline \text{进位 } 1111111 \\ 100010010 \end{array}$$

$$(10110111)_2 + (01011011)_2 = (100010010)_2$$

例1.3 二进制减法举例：求 $(10110111)_2 - (01011011)_2$ 的差。

$$\begin{array}{r} \text{借位 } 1011000 \\ 10110111 \\ - 01011011 \\ \hline 01011100 \end{array}$$

$$(10110111)_2 - (01011011)_2 = (01011100)_2$$

2. 八进制数

- 八进制数的特点：

- (1) 有0, 1, 2, 3, 4, 5, 6, 7八个不同的数字符号。

- (2) 基本运算规则：逢8进位1。

- 八进制数是由0到7八个数码组成的数字串，一般书写表示如：(136)₈

- C语言中八进制数的表示：加前导零。

如：0136，表示的是八进制数(136)₈。

3. 十六进制数及其运算

- 十六进制数的特点:

(1) 有0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F十六个不同的数字符号 (A~F也可以采用小写英文字母a~f, 代表十进制的10~15)。

(2) 基本运算规则: **逢16进位1**。

- C语言中十六进制数的表示: 加前导**0x**或**0X**

如: **0x2d5b**

0XABCD

疑问: C语言中二进制数是如何表示的?

C语言中不能直接用二进制数

表1. 2十六进制的16个不同数码与十进制、八进制、二进制数之间的关系

十进制	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
二进制	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
八进制	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

注：为了表示十进制0~15范围的数，二进制下至少用4位表示：0000~1111

◆ 八进制数、十六进制数的算术运算

例1.4 八进制加减法运算

见教材P10.

要点：加法 “逢八进一”、减法 “借一抵八”

例1.5 十六进制加减法运算

$$0XABCD + (2d5b)_{16} = 0XD928$$

$$0XABCD - (2d5b)_{16} = 0x7E72$$

要点：加法 “逢十六进一”、减法 “借一抵十六”

1.5.2 不同进位制数之间的转换

1. 十进制数转换为二进制数

1) 十进制整数转换为二进制整数：除2取余法

例1.6 用除2取余法求十进制整数 189 的二进制表示。

解

$2 \overline{) 189}$	余 1 (最低位)
$2 \overline{) 94}$	余 0
$2 \overline{) 47}$	余 1
$2 \overline{) 23}$	余 1
$2 \overline{) 11}$	余 1
$2 \overline{) 5}$	余 1
$2 \overline{) 2}$	余 1
$2 \overline{) 1}$	余 0
0	余 1 (最高位)

$(189)_{10} = (10111101)_2$

高 低

2) 十进制纯小数转换二进制小数：乘基取整法

如：将 $(0.6875)_{10}$ 转换为二进制小数表示

$$0.6875 \times 2 = 1.375$$

$$0.375 \times 2 = 0.75$$

$$0.75 \times 2 = 1.5$$

$$0.5 \times 2 = 1$$

1 高位

0

1

1 低位

$$(0.6875)_{10} = (0.1011)_2$$

高 低

3) 带有整数部的十进制小数转换成二进制数形式

整数部和小数部分别转换，然后合在一起即可

$$\text{如： } (189.6875)_{10} = (10111101.1011)_2$$

2. 二进制数转换为十进制数

- 类似于十进制的"个、十、百、千"等，二进制数中的各个数位也具有不同的权值：

.....	2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}	
	第5位	第4位	第3位	第2位	第1位	小数点	第1位	第2位	第3位	第4位	

◆ 二进制数转换为十进制数：按权展开

$$\begin{aligned} \text{如: } (10111101.1101)_2 &= 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + \\ &\quad 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + \\ &\quad 0 \times 2^{-3} + 1 \times 2^{-4} = (189.8125)_{10} \end{aligned}$$

3. 二进制整数转换为八进制整数: 3位分组法

对二进制数，从整数部分的最低位起，**每3位分成1组**，不足3位加前导0补足3位，然后**把每3位二进制数用一个对应的八进制数码来表示**即可。

例1.8

$$(10111101)_2 = (\mathbf{010\ 111\ 101})_2 = (275)_8$$

加前导0，补足3位

4. 八进制整数转换为二进制整数：一变三

对八进制整数，将**每一位八进制数用相同大小的3位二进制数**表示即可。

例1.9

$$\begin{aligned}(171777)_8 &= (\mathbf{001\ 111\ 001\ 111\ 111\ 111})_2 \\ &= (1111\mathbf{00}1111111111)_2\end{aligned}$$

5. 二进制整数转换为十六进制整数： **4位分组法**

对二进制整数，从整数部分的最低位起，**每4位分成1组**，不足4位时加前导0补足4位，然后**把每4位二进制数用一个对应大小的十六进制数码来表示**。

例1.10

$$\begin{aligned} & (10111101)_2 \\ &= (1011 \ 1101)_2 \\ &= (BD)_{16} \end{aligned}$$

6. 十六进制整数转换为二进制整数：一变四

对十六进制整数，将每一个十六进制数码用相同大小的**4位二进制数**表示即可。

例1.11

$$\begin{aligned} (ABCD)_2 &= (1010 \ 1011 \ 1100 \ 1101)_2 \\ &= (1010101111001101)_2 \end{aligned}$$

- 上述方法表示出了数的大小
- 还有一个问题没有解决：？ **符号**

进一步需要解决的问题：

数有正负之分，怎么表示正负呢？

二进制数的正负表示：引入**符号位**

- 对一个n位数，令最高位为符号位，其余n-1位为数值位。

符号位为**0**代表正数，**1**代表负数。

如：四位数**0100** 表示+4，**1100**表示-4

八位数**00001111** 表示+15，**10001111**表示-15

- 带符号位的数称为**带符号数**，简称**符号数**；
- 不带符号位的数称为**无符号数**。
 - ◆ 无符号数没有符号位，**所有的n位都作为数值位对待，其值被视为正数：**

如：四位数**0100**：带符号数表示+4，无符号数也表示+4

1100：带符号数表示-4，无符号数表示+12

八位数**00001111**：带符号数表示+15，无符号数也表示+15

10001111：表示带符号数表示-15，无符号数表示+143

◆ 无符号数表示的正整数的范围比带符号数表示的正整数范围大一倍。但能够表示的数字的个数是一样的，都为 2^n 个

◆ 为什么会有无符号类型的数？

有些数据是不用考虑负数的，比如：

- 表示数量个数的数据：现实世界中没有-1个之说
- 内存地址：内存空间字节位置的编号，从0开始，递增编址即可

所以，这一类的数据就可以定义成无符号数。

【好处：相比长度相同的有符号数，无符号数表示的非负数据范围大一倍】

1.5.3 数的机器码表示

1. 数的真值

记一个数 x 的绝对值为 X 。若 x 为正数，以“ $+X$ ”形式表示；若 x 为负数，以“ $-X$ ”形式表示。这种以“ $\pm$ 绝对值”形式表示的数称为该数的真值。

如： $+11101$ ， -1011.0101

- 其中，正号可以省略，如 11101 等价于 $+11101$ ；
- 这里视 x 、 X 均为二进制形式。

2. 机器数

机器数：数在**计算机内部的表示**称为该数的**机器数**。

通常，机器数形式 $\neq$ 真值形式

那是一种什么样的形式呢？

1) 机器数的长度

设一个二进制数 x 有 n 位，如 $x=11001$ 有5位。由于计算机中**位 (bit)** 不能单独存取，所以，一般不是用 n -bits直接表示 x ，而是用**总bit数**不少于 n 的若干字节表示（其中的低 n 位用于表示 x ，多出的位填0或符号位）。

如： $x=(11001)_2$ ，则至少用1个字节（8bit_s）表示，形式如下：

0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---

若用2字节（16bit_s）表示，则字节内容为

0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

数的长度：一个数在计算机中表示时使用的字节数，称为这个**数的长度**。

- ◆ 数的长度与其**数据类型**相关，以后讲解；

2) 数在计算机中的表示：机器数

这里需要解决两个问题：

- (1) **正负号的表示**：指定用数所占字节的最高位表示，并约定最高位为0表示正数，1表示负数：



正数



负数

- (2) **值(大小)的表示**：与编码方式有关(原码、反码、补码等)

机器数有原码、反码、补码等多种形式：

(1) **原码**：原码是机器数的一种

对于一个二进制数，令**最高位为符号位**，**其余各位为该数的绝对值**，数的这种**编码**称为数的**原码**。

例1.12 求十进制数125和-125的8位、16位原码表示。

解： $(125)_{10} = 0X7D = (+1111101)_2$ ， $(-125)_{10} = -0X7D = (-1111101)_2$

则根据原码的定义有：



$$[+1111101]_{\text{原}} = \mathbf{0}1111101 \quad (8\text{位})$$

$$[+1111101]_{\text{原}} = \mathbf{0}000000001111101 \quad (16\text{位})$$

$$[-1111101]_{\text{原}} = \mathbf{1}1111101 \quad (8\text{位})$$

$$[-1111101]_{\text{原}} = \mathbf{1}000000001111101 \quad (16\text{位})$$

■ 原码的优点：

- 容易换算出其真值：首位决定符号，其余位是其绝对值

■ 原码的缺点：

- 真值0的原码表示不唯一，有+0和-0之分：

$$[+0]_{\text{原}} = 00000000 \quad (8\text{位})$$

$$[-0]_{\text{原}} = 10000000 \quad (8\text{位})$$

- 不利于减法运算的实现

(2) 补码

补码的定义：正数的补码等于其原码；

负数的补码为其原码**除符号位不动，其余各位变反，然后整体加1**所得。

例1.13 求十进制数35和-1在8位机和16位机中的补码表示。

注，8位机中整数用1个字节表示，16位机中整数用2个字节表示

解：设 $x_1=35$ ， $x_2=-1$ ；

16位机自己试一下

以8位机为例：

$$\because x_1 > 0, \therefore [x_1]_{\text{补}} = [x_1]_{\text{原}} = 00100011;$$

$$\because x_2 < 0, \therefore [x_2]_{\text{原}} = 10000001,$$

$$[x_2]_{\text{补}} = 11111110 + 1 = 11111111;$$

补码的优点

- 零的补码表示形式唯一：

$$[+0]_{\text{补}} = [-0]_{\text{补}} = 000\dots000$$

注：最右边式子中0的个数等于机器的字长。

- 在整数减法中，“减一个数”等于“加上这个负数的补码”
(即 $x - y \equiv x + (-y)_{\text{补}}$)，从而将减法运算转化成补码的加法运算，简化了加减法运算电路的实现（具体原理以后再学）。

(3) 反码

反码：对正数而言，其反码与其原码相同；

对负数而言，反码符号位与原码相同，数值是对原码的各数值位**逐位变反**所得。

例1.14 分别求十进制数6和-1在8位机中的反码表示。

解： $[6]_{\text{反}} = 00000110$ ；

$[-1]_{\text{反}} = \text{对}[10000001]_{\text{原}}\text{的数值位按位变反}$
 $= 11111110$

注：负数的补码=负数的反码+1

3. 各种计算机编码

- **码**是指计算机中数据表示的抽象表述。
- 根据数据的性质，码可以分为**数值码**、**字符码**和**图象码**等。
 - **数值码**：用于表示数值型数据
 - **字符码**：用于表示字符型数据
 - **图像码**：用于表示图像数据
 - **音频码**：用于表示音频数据

不管是哪种数据编码，在计算机中都是用0/1形式的**二进制码的基本形态**存储在内存之中的。**怎么理解和使用一串0/1表示的数据取决于应用程序赋予的逻辑含义。**

1.5.4 字符的编码表示

字符数据：是指英文字母、标点符号、控制字符，以及中文、日文、阿拉伯语等各民族的文字符号。

- ◆ 我们在屏幕上看到的是字符的“图形”；
- ◆ 但计算机中存储的不是字符的“图形”，而是其编码。

字符编码：计算机中有一个“码表”，还有一个“字库”。每个字符在“码表”里都有一个唯一指定的编号，知道编号就知道是哪个字符，这个编号就是**字符编码**；

字形：“字库”里则有每个符号的“图形”信息，根据字库中符号的**图形描述**，就可以在屏幕或打印机上“画”出符号的形状。

1. 西文字符的国际标准编码——ASCII码

西文字符，英文字母、阿拉伯数字0~9、英文标点符号、控制字符，国际上采用的编码是**ASCII码**（American Standard Code for Information Interchange，美国国家信息交换标准字符码）。

标准ASCII码采用**单字节编码**（8bit_s），且只用**低7位编码**（最高位留做校验位），所以共有**128个字符**的编码。

- ◆ 编码范围0~127，每个西文字符在ASCII码字符表中都有一个唯一的编号，这个编号称为该字符的**ASCII码**。
- ◆ 字符和编码一一对应，128个编码对应128个字符。

IBM PC 的 ASCII 碼

ASCII 值			ASCII 值			ASCII 值			ASCII 值		
十進位	十六進位	字元符號	十進位	十六進位	字元符號	十進位	十六進位	字元符號	十進位	十六進位	字元符號
000	00	(null)	032	20		064	40	@	096	60	
001	01	☺	033	21	!	065	41	A	097	61	a
002	02	●	034	22	"	066	42	B	098	62	b
003	03	♥	035	23	#	067	43	C	099	63	c
004	04	♦	036	24	\$	068	44	D	100	64	d
005	05	♣	037	25	%	069	45	E	101	65	e
006	06	♠	038	26	&	070	46	F	102	66	f
007	07	·	039	27	'	071	47	G	103	67	g
008	08	▣	040	28	(	072	48	H	104	68	h
009	09	○	041	29	)	073	49	I	105	69	i
010	0A	▣	042	2A	*	074	4A	J	106	6A	j
011	0B	♂	043	2B	+	075	4B	K	107	6B	k
012	0C	♀	044	2C	,	076	4C	L	108	6C	l
013	0D	♪	045	2D	-	077	4D	M	109	6D	m
014	0E	♫	046	2E	.	078	4E	N	110	6E	n
015	0F	*	047	2F	/	079	4F	O	111	6F	o
016	10	▶	048	30	0	080	50	P	112	70	p
017	11	◀	049	31	1	081	51	Q	113	71	q
018	12	‡	050	32	2	082	52	R	114	72	r
019	13	!!	051	33	3	083	53	S	115	73	s
020	14	¶	052	34	4	084	54	T	116	74	t
021	15	§	053	35	5	085	55	U	117	75	u
022	16	-	054	36	6	086	56	V	118	76	v
023	17	‡	055	37	7	087	57	W	119	77	w
024	18	↑	056	38	8	088	58	X	120	78	x
025	19	↓	057	39	9	089	59	Y	121	79	y
026	1A	→	058	3A	:	090	5A	Z	122	7A	z
027	1B	←	059	3B	;	091	5B	[	123	7B	{
028	1C	↵	060	3C	<	092	5C	\	124	7C	
029	1D	↔	061	3D	=	093	5D	]	125	7D	}
030	1E	▲	062	3E	>	094	5E	^	126	7E	~
031	1F	▼	063	3F	?	095	5F	_	127	7F	■

【1】前32个码 (0x00到0x1f) 和最后一个码 (0x7f) 为控制码, 如控制回车、换行、振铃、空等;

【2】从0x20到0x7e共95个编码, 对应数字字符0~9, 大小写英文字母a~z和A~Z, 以及标点符号。

【3】计算机的终端 (键盘、显示器) 只能输入和显示这95个编码对应的字符。同时, 打印机也只能打印输出这95个编码对应的字符。完整的ASCII码字符集请参阅附录1。

ASCII码			ASCII码			ASCII码			ASCII码		
十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符
128	80	Š	160	A0	á	192	C0	Ł	224	E0	α
129	81	ü	161	A1	í	193	C1	ł	225	E1	β
130	82	é	162	A2	ó	194	C2	ŧ	226	E2	γ
131	83	â	163	A3	ú	195	C3	ţ	227	E3	π
132	84	ä	164	A4	ñ	196	C4	—	228	E4	Σ
133	85	à	165	A5	ñ	197	C5	†	229	E5	σ
134	86	å	166	A6	ª	198	C6	‡	230	E6	μ
135	87	ç	167	A7	º	199	C7	‡	231	E7	τ
136	88	ê	168	A8	¿	200	C8	Ł	232	E8	φ
137	89	ë	169	A9	ƒ	201	C9	Ŕ	233	E9	θ
138	8A	è	170	AA	¬	202	CA	Ł	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	Ŧ	235	EB	δ
140	8C	î	172	AC	¼	204	CC	‡	236	EC	∞
141	8D	ì	173	AD	¡	205	CD	=	237	ED	φ
142	8E	Ä	174	AE	«	206	CE	‡	238	EE	€
143	8F	Å	175	AF	»	207	CF	≡	239	EF	∩
144	90	É	176	B0	⋮	208	D0	Ł	240	F0	≡
145	91	æ	177	B1	⋮	209	D1	Ŧ	241	F1	±
146	92	Æ	178	B2	⋮	210	D2	Ŧ	242	F2	≥
147	93	ô	179	B3		211	D3	Ł	243	F3	≤
148	94	ö	180	B4	†	212	D4	Ł	244	F4	┘
149	95	ò	181	B5	‡	213	D5	Ŕ	245	F5	┘
150	96	û	182	B6	‡	214	D6	Ŕ	246	F6	÷
151	97	ù	183	B7	ŧ	215	D7	‡	247	F7	≈
152	98	ÿ	184	B8	ŧ	216	D8	‡	248	F8	°
153	99	Ö	185	B9	ŧ	217	D9	ŧ	249	F9	•
154	9A	Ü	186	BA		218	DA	Ŕ	250	FA	·
155	9B	Ŧ	187	BB	ŧ	219	DB	■	251	FB	√
156	9C	£	188	BC	ŧ	220	DC	■	252	FC	ⁿ
157	9D	¥	189	BD	ŧ	221	DD	■	253	FD	²
158	9E	₪	190	BE	ŧ	222	DE	■	254	FE	■
159	9F	ƒ	191	BF	ŧ	223	DF	■	255	FF	

扩展ASCII码：由80H到FFH的**后128个字符**称为扩展ASCII码。由IBM制定，许多基于x86的系统都支持使用扩展ASCII。

扩展ASCII 码允许将每个字符的第8位用于确定附加的128个特殊符号，包括音标、外来语字母和图形符号等。

2. 汉字编码

汉字编码有许多方式。常用的有：国标码、区位码、机内码、输入码等。

- 1) 输入码：用于汉字输入的编码，如：拼音码和五笔字型码等。
- 2) 国标码：是我国于1981年颁布的《通用汉字字符集(基本集)及其交换码标准》规定的汉字字符编码方案，标准编号是GB2312-80，总共收集了7445个汉字、字母、数字和符号。

3) 区位码:

- ◆ 在国标GB2312—80中规定, 所有的国标汉字及符号分配在一个**94行X94列**的方阵中, 方阵的每一行称为一个“**区**”, 编号为01~94区; 每一列称为一个“**位**”, 编号为01~94位。
- ◆ 方阵中的每一个汉字和符号所在的区号和位号组合在一起形成的**四个阿拉伯数字码**就是它们的“**区位码**”。

Diagram illustrating the GB2312-80 character matrix structure. A bracket on the left labeled "区" (Area) spans the rows, and a bracket on top labeled "位" (Position) spans the columns. The matrix is a 94x94 grid of characters and their corresponding zone and position codes.

(0101)	· (0102)	ˆ (0103)	• (0104)	ˆ (0105)	ˆ (0106)	ˆ (0107)	ˆ (0108)	ˆ (0109)	ˆ (0110)
ˆ (0111)	ˆ (0112)	ˆ (0113)	ˆ (0114)	ˆ (0115)	ˆ (0116)	ˆ (0117)	ˆ (0118)	ˆ (0119)	ˆ (0120)
ˆ (0121)	ˆ (0122)	ˆ (0123)	ˆ (0124)	ˆ (0125)	ˆ (0126)	ˆ (0127)	ˆ (0128)	ˆ (0129)	ˆ (0130)
ˆ (0131)	ˆ (0132)	ˆ (0133)	ˆ (0134)	ˆ (0135)	ˆ (0136)	ˆ (0137)	ˆ (0138)	ˆ (0139)	ˆ (0140)
ˆ (0141)	ˆ (0142)	ˆ (0143)	ˆ (0144)	ˆ (0145)	ˆ (0146)	ˆ (0147)	ˆ (0148)	ˆ (0149)	ˆ (0150)
ˆ (0151)	ˆ (0152)	ˆ (0153)	ˆ (0154)	ˆ (0155)	ˆ (0156)	ˆ (0157)	ˆ (0158)	ˆ (0159)	ˆ (0160)
ˆ (0161)	ˆ (0162)	ˆ (0163)	ˆ (0164)	ˆ (0165)	ˆ (0166)	ˆ (0167)	ˆ (0168)	ˆ (0169)	ˆ (0170)
ˆ (0171)	ˆ (0172)	ˆ (0173)	ˆ (0174)	ˆ (0175)	ˆ (0176)	ˆ (0177)	ˆ (0178)	ˆ (0179)	ˆ (0180)
ˆ (0181)	ˆ (0182)	ˆ (0183)	ˆ (0184)	ˆ (0185)	ˆ (0186)	ˆ (0187)	ˆ (0188)	ˆ (0189)	ˆ (0190)
ˆ (0191)	ˆ (0192)	ˆ (0193)	ˆ (0194)	ˆ (0201)	ˆ (0202)	ˆ (0203)	ˆ (0204)	ˆ (0205)	ˆ (0206)
ˆ (0207)	ˆ (0208)	ˆ (0209)	ˆ (0210)	ˆ (0211)	ˆ (0212)	ˆ (0213)	ˆ (0214)	ˆ (0215)	ˆ (0216)
ˆ (0217)	ˆ (0218)	ˆ (0219)	ˆ (0220)	ˆ (0221)	ˆ (0222)	ˆ (0223)	ˆ (0224)	ˆ (0225)	ˆ (0226)
ˆ (0227)	ˆ (0228)	ˆ (0229)	ˆ (0230)	ˆ (0231)	ˆ (0232)	ˆ (0233)	ˆ (0234)	ˆ (0235)	ˆ (0236)

◆ 在区位码表里, 每个汉字或符号都有一个唯一的区位码

◆ 区位码的前两位是它的**区号**, 后两位是它的**位号**。

如: 汉字“母”字的区位码是3624_(十进制), 表明它在方阵的36区24位

- 国标码和区位码之间的换算关系：

国标码（十六进制） = 区位码（十六进制） + 0x2020

如：汉字“母”字的区位码是：0x2418

则其国标码是：0x4438

◆ 汉字采用两字节编码方案

- 中文国标字符集有7445个汉字和中文符号
- 区位码、国标码都是**二字节码**，两个字节分别表示“区”和“位”，用字节的低七位进行编码。
- ◆ **编码长度**： 16bit（两字节）
- ◆ **编码容量**： $2^{16} = 65536$ 个字符
- ◆ **实际编码数**： $128 * 128 = 16384$ 个字符

4) 汉字的机内码:

- ◆ 机内码是计算机内部存储和处理汉字时采用的汉字编码。
- ◆ 机内码是将国标码两个字节的**每个字节的最高位置“1”**后形成（为什么？为了与基本ASCII相互区分）。
- ◆ 机内码与国标码之间的转换关系是：

$$\text{机内码 (十六进制)} = \text{国标码 (十六进制)} + 0x8080$$

例如：中国的“中”字的机内码是0xd0d6 (+0x8080)

国标码是0x5056 (+0x2020)

区位码是0x3036



课后作业：

3. 回答一下问题：

(1) 汉字的区位码、国标码、机内码是什么？它们之间的转换关系式什么？

(2) 字库是什么？点阵字库中的字模的作用是什么？

(3) 什么是点阵字库？什么是矢量字库？

(4) 汉字是如何输入计算机的？计算机内存的汉字信息是什么？汉字是如何显示在屏幕上的？

1.6 算法及其表示（先自学，以后再回顾）

算法是指为解决某个问题所采取的方法和步骤。

算法的表示方式多种多样，如自然语言方式，流程图方式、表格方式、伪代码方式等。

- 自然语言方式非常容易理解

- 但是在将自然语言描述的算法转换为具体程序方面不够直观，它最适合表达算法的初始设计思想。

- 流程图方式非常直观，易于将它描述的算法转换成程序

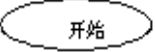
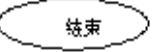
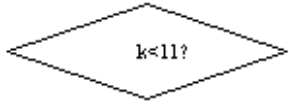
- 但是在程序规模变大的情况下，用流程图来完整描述算法不仅多余，而且不太可能。因此，流程图方式适合初学者。本书重点介绍流程图方法。

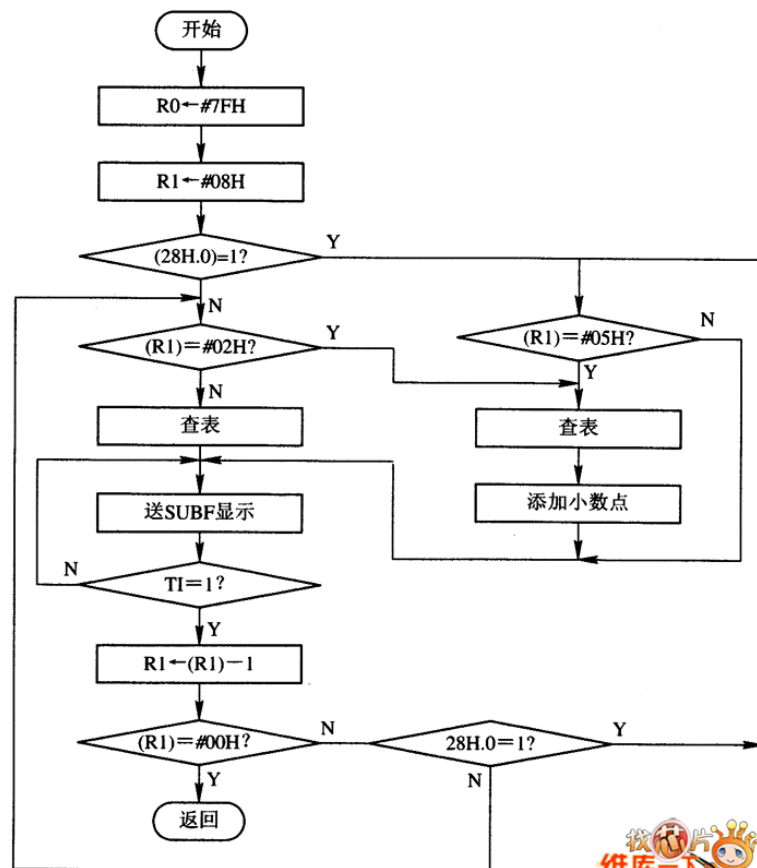
- 伪代码方式：算法课程里学习

1. 流程图的图符

流程图又称为框图。

它是用它的图符来表示算法的每一步骤以及这些步骤之间的相互联系。

图 符	名 称	功 能	例 子
	起止框	表示算法的开始或算法的结束	 
	处理框	表示按顺序执行操作步骤	sum=0; k=1;
	判断框	表示判断选择。根据框中列出的二值判断条件满足与否，从两种备选步骤中选择其一执行	
	流程线	连接两个相邻的步骤。表示执行顺序从从矢尾向箭头。	Y: Yes, N: No <pre> graph TD Start([开始]) --> J1{k<11?} J1 -- Y --> P1[sum=sum+k; k++] P1 --> J1 J1 -- N --> P2[输出 sum] P2 --> End([结束]) </pre>
	连接点	表示一条流程线被断开后的两个端点。连接点必须相同的形式成对出现。	<pre> graph TD J2{k<11?} -- Y --> P3[sum=sum+k; 输出 sum] P3 --> End2([结束]) J2 -- N --> C1((1)) C2((1)) --> P4[输出 sum] P4 --> End2 </pre>



2. 用流程图描述算法

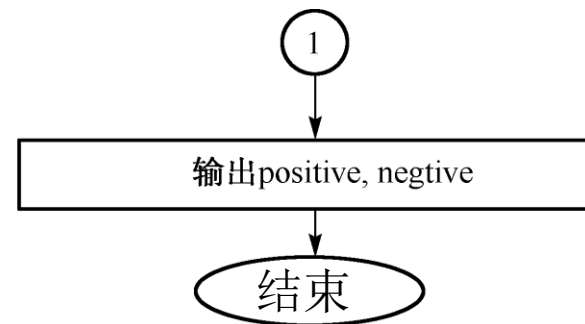
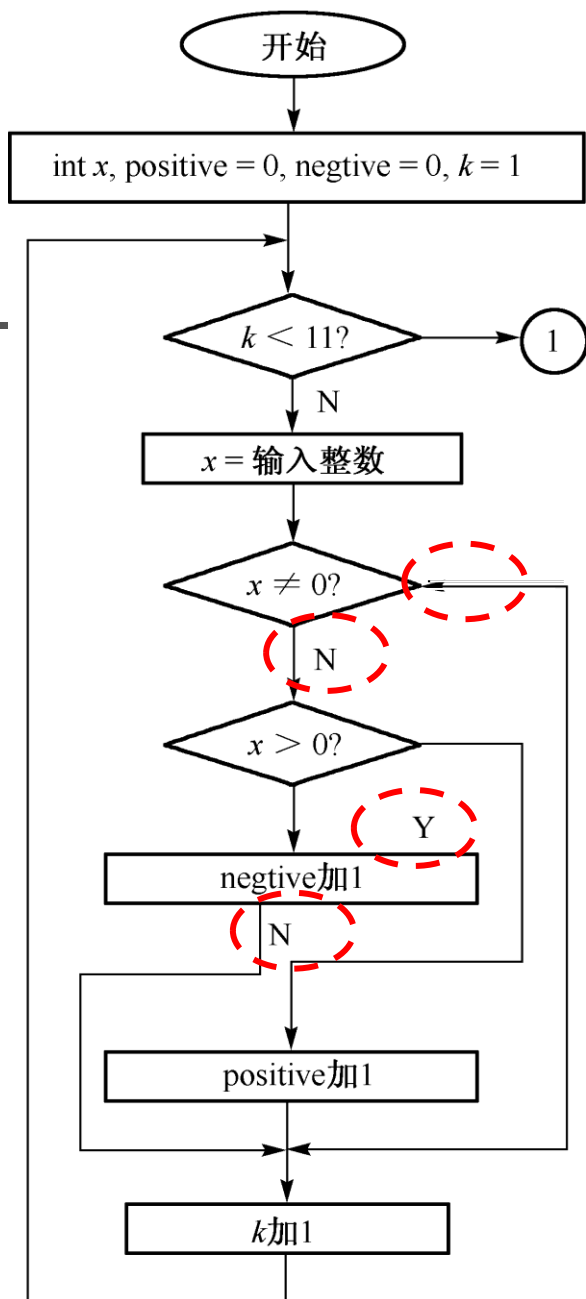
问题描述：输入10个整数，统计正数和负数的个数。

算法设计：用变量x存储输入的整数，k表示输入的次数，变量positive表示正数的个数，变量negative表示负数的个数。

用自然语言描述

- (1) 定义变量x, k, positive, negative。
- (2) positive, negative清零, k赋初值1。
- (3) 判断 $k < 11$ 吗? (或 $k \leq 10$) , 如果是, 顺序执行步骤 (4) ; 否则转步骤 (10) 。
- (4) 从键盘输入一个整数并且赋给变量x。
- (5) 判断x是否等于零? 如果是, 转步骤 (9) ; 否则顺序执行步骤 (6) 。
- (6) 判断x是否大于零? 如果是, 转步骤 (8) ; 否则顺序执行步骤 (7) 。
- (7) 负数个数negative加1; 转步骤 (9) ;
- (8) 正数个数positive加1; 顺序执行步骤 (9) 。
- (9) 次数k加1, 转步骤 (3) 。
- (10) 输出正数个数positive, 负数个数negative。
- (11) 结束。

算法的流程图描述



1. 6. 3 算法的实现

```
#include <stdio.h>
```

```
int main(void)           // 下面的main为主函数名，括号中的void表示无参
```

```
{ // main函数的函数体开始
```

```
    int x,k,positive,negative;      /* 说明语句说明各个整型变量 */
```

```
    k=1;                            /* 循环次数变量k赋初值1 */
```

```
    positive=negative =0;          /* 具有累加作用的变量清零 */
```

```
    while(k<11)                    /* 如果k<11，执行while循环体，否则转到循环体后一语句执行 */
```

```
    { /* while循环体开始 */
```

```
        scanf("%d",&x);            /* 接受从键盘输入的一个整数，并赋给变量x */
```

```
        if(x!=0)                    /* 若x≠0，执行外层if块，否则转执行外层if块后一语句执行 */
```

```
        { /*外层if块开始*/
```

```
            if(x>0)                /* 如果x>0，执行内层if块，否则执行else块 */
```

```
            { /* 内层if块开始 */
```

```
                positive++;         /* 正整数个数加1 */
```

```
            } /* 内层if块结束 */
```

```
            else /* 如果x<0，转到此处执行 */
```

```
            { /* else块开始*/
```

```
                ++negative;         /* 负整数个数加1 */
```

```
            } /* else块结束 */
```

```
        } /* 外层if块结束 */
```

```
        k++; /* 循环次数变量k加1 */
```

```
    }
```

```
    printf("positive=%d \n",positive); /* 输出正数个数*/
```

```
    printf("negative=%d\n",negative); /* 输出负数个数*/
```

```
    return 0;
```

```
} // main函数的函数体结束
```

输入10个整数及程序的运行结果如下：

 e:\test\bin\Debug\test.exe

```
8
0
-3
-5
-7
5
9
-16
20
6
positive=5
negative=4

Process returned 0 (0x0)    execution time : 46.978 s
Press any key to continue.
```


1.7 学习C语言与程序设计的方法

现在还是菜鸟的你，如何学会、掌握、以及熟练使用C语言？

从四个方面进行学习和训练：

首先，要学习并理解C语言的语法和语义；

其次，要学习并掌握一些基本数据结构和常用算法的设计；

第三，要学习并熟悉与C语言的集成开发环境；

第四，要熟悉C语言提供的各种库函数。

1) 学习并理解C语言的语法和语义——文字正确

- **语义**：在C语言中，语义指的是构成C语言的**各种符号的含义**。

由于C程序是由C语言的符号所组成，因此只有准确理解各种符号的语义，才有可能使用各种合适的符号来表达自己的编程思想。

- **语法**：C的语法规定了C语言符号集中**各种符号的结合方式**，规定了各种表达式、语句、函数，乃至程序结构等的**构造规则**。

只有理解和掌握C语言的语法，才可能正确地用各种符号去构造程序的各种成分，并进而构成满足算法要求、能解决实际应用问题的程序。

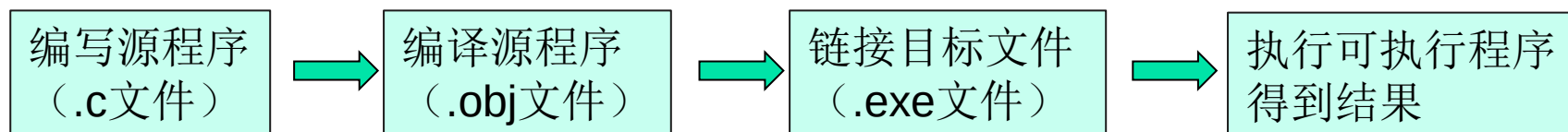
2) 学习并掌握一些基本数据结构和常用算法的设计

——逻辑正确

- C语言对机器硬件有很强的操纵能力，在大多数情况下可以代替汇编语言。因此，要学好用好C，需要对字符、整型变量、数组元素在内存中的存储形式，对原码、补码、字节中的位（bit）的操作做到熟练掌握。
- 深刻理解各类指针及所指对象的声明、使用以及物理含义。对结构，结构数组、链表都要熟练掌握。
- 由于文本是C语言的基本研究对象，对文本操作的各种算法，如统计字符、单词的个数，字符串的各种操作所涉及的算法要熟练掌握。
- 熟练掌握各种算法，包括查找、排序、枚举、递推、迭代、分治、贪心、递归、回溯、动态规划、高精度计算等，都应该熟练掌握。这样才能奠定程序设计的扎实基础。

3) 熟悉集成开发环境——掌握编程技术

- 首先，用文本编辑器创建源程序，也称为源文件，C语言的源程序一般命名为.c文件。
- 然后，对源程序进行编译，产生目标程序文件，一般为.obj文件。
- 再后，使用链接程序Link进行链接，生成可执行程序，一般为.exe文件。
- 最后，将可执行程序投入运行以得到运行结果。



几乎所有的程序设计语言都有自己的**集成开发环境**（IDE）。集成开发环境将源程序的**创建**、**编辑**，以及其后的**编译**、**链接**，乃至程序的调试与跟踪全部都集成于一体。方便编程者开发程序。

编程者只有学会使用集成开发环境，才能进行更快、更好地进行程序的设计与开发。

本课程使用的IDE：**Code::Blocks**、**Dev C++**、**VS**等。

4) 熟悉库函数——提高编程效率

在C语言中，编译系统提供了一些标准的库函数。

如：输入/输出标准库函数

字符串处理标准库函数

数学计算标准库函数等。

这些函数是C语言的
设计者帮你写好的，
你只管用

编程者只要知道这些标准库函数的功能、调用方式、返回何种结果，并且能够在自己的程序中调用这些标准库函数就行了，没有必要自己去编写这些函数。

编程者还可以根据自己的需要创建自己的函数和函数库。

1.8 着手编程：第一个程序例子

准备工具环境：

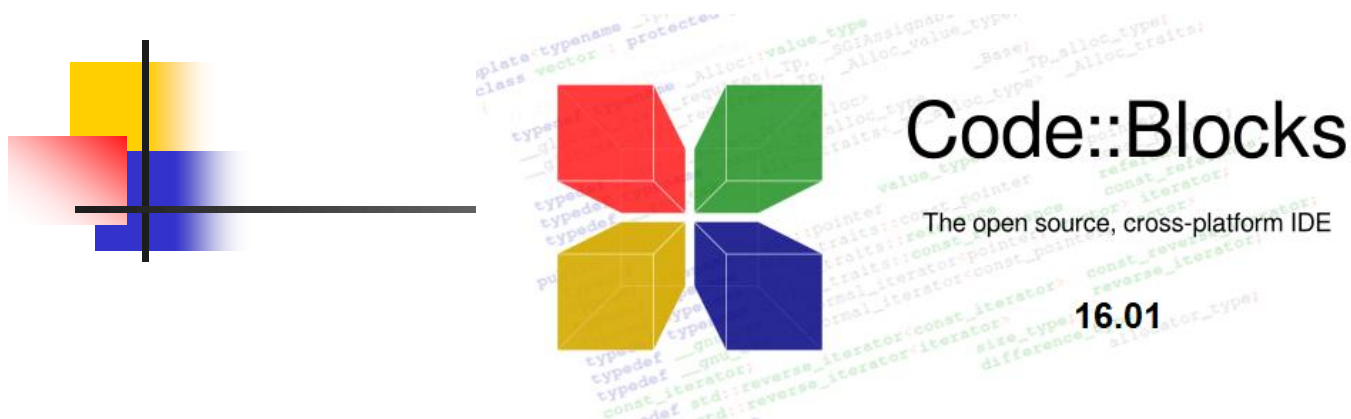
- (1) 一台安装了Windows 操作系统的电脑（其它OS自备）
- (2) 安装编程工具软件Code::Blocks

Code::Blocks官网：<http://www.codeblocks.org/>

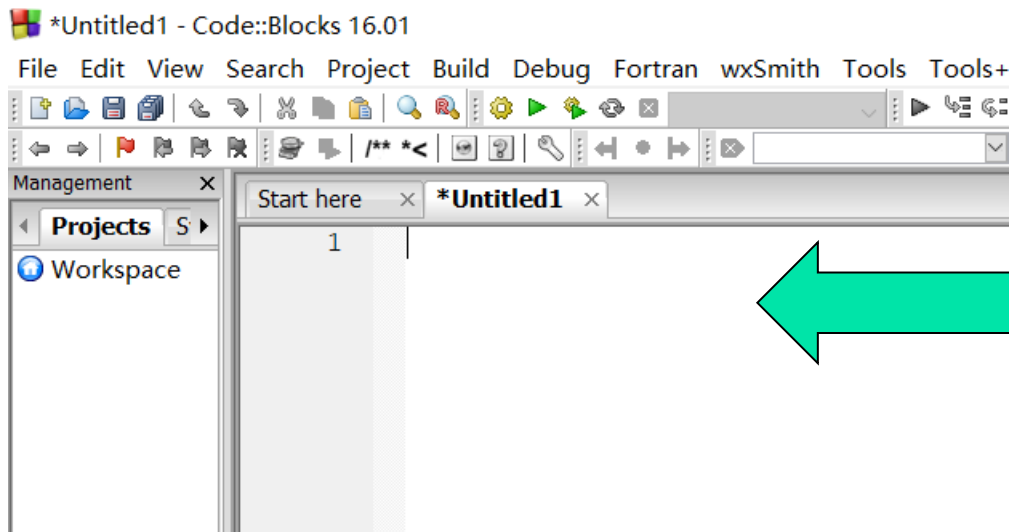
下载安装包，安装Code::Blocks，一定要**安装GCC编译器**



执行Code::Blocks



主菜单→File →New →Empty File: 建立一个空文件



在编辑区输入
下面的程序:

例1.1 输入你自己的名字的汉语拼音，要计算机问候自己并且输出这是自己学习C语言的第一个程序的句子。

```
#include <stdio.h>
```

(源程序\ex1_1.c)

```
void show(char str[]);
```

```
int main(void)
```

```
{
```

```
    char name[20];
```

```
    printf("Input your name please!\n");
```

```
    gets(name);
```

```
    printf("Hello %s!\n",name);
```

```
    show(name);
```

```
    return 0;
```

```
}
```

```
void show(char str[])
```

```
{
```

```
    printf("This is the first program for %s to learn C programming!\n",str);
```

```
}
```

对程序的说明

1. `#include <stdio.h>`

【1】 `#include`: C语言包含命令, 以#号开始, 后跟称为“头文件”的磁盘文件名, 用双引号括起来, 如“`stdio.h`”

2. `void show(char str[]);`

【2】 `show`函数的函数声明

3. `int main(void)`

【3】 `main`函数的定义

```
4. {  
5.     char name[20];  
6.     printf("Input your name please!\n");  
7.     gets(name);  
8.     printf("Hello %s!\n",name);  
9.     show(name);  
10.    return 0;  
11. }
```

12. `void show(char str[])`

【4】 `show`函数的定义

```
13. {  
14.     printf("This is the first program for %s to learn C programming!\n",str);  
15. }
```

C语言基础:

【1】什么是包含命令? `#include <stdio.h>`的作用是什么?

【2】函数的基本概念

“函数”引申至数学里的概念

例, 设有数学函数 $f(x,y)=x+y$

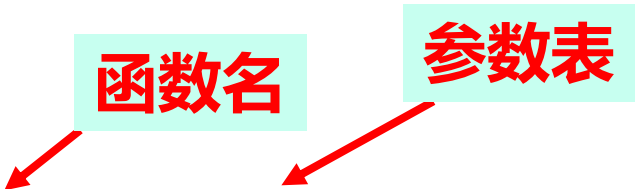
$x+y$: 函数的计算公式, 称为函数体
函数体说明函数执行什么运算

这里, f : 函数名; (x,y) : 参数表, x 、 y : f 的参数

设有变量 z , 令 $z = f(2,3)$

这里, 表达式 $z = f(2,3)$ 称为以2和3参数调用 f 函数, 得到 f 的计算结果赋给 z , 最后的结果是 $z=5$ 。

借此，C语言将按照以下形式定义的程序段也称为函数



```
void show(char str[])  
{  
    printf("This is the first program for %s to learn C programming!\n",str);  
}
```



函数体：一对花括号之间的程序段称为函数体
函数体说明该函数的功能。

- ◆ 自己定义的函数叫做“**用户自定义函数**”，如show函数；
- ◆ 有些函数是C语言提供的，不需要用户再自己编写，这类函数称为“**系统函数**”或“**库函数**”，用户只要调用使用即可。

➤ **这里用到两个库函数**

▣ **printf(...)函数**：将参数表里双引号里面的内容输出至屏幕

如：printf("Input your name please!\n");

▣ **gets(...)函数**：接收用户通过键盘输入一个字符串，并把输入的字符串保存到参数数组中。

如：gets(name);

main函数是一个特殊的函数，每个C程序都必须有且仅有一个main函数，它是整个程序的“入口点”。

如果函数有返回值，需要在这里指出函数返回值的类型，这里int表示main函数返回整型数值

int main(void)

{

```
char name[20];  
printf("Input your name please!\n");  
gets(name);  
printf("Hello %s!\n",name);  
show(name);  
return 0;
```

}

main也是一个函数，参数表里写void表示它不需要参数

main函数的函数体

main函数有返回值，通过return语句将结果“返回”

show函数是用户自定义函数；没有返回值，所以函数体里没有return语句，同时函数名前写上**void**表示该函数没有返回值



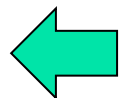
```
void show(char str[])
```

```
{  
    printf("This is the first program for %s to learn C programming!\n",str);  
}
```



函数体里没有return语句，表示该函数没有返回值

```
1. #include <stdio.h>
2. void show(char str[]);
```



如果程序在书写时，一个函数在调用之后才定义，则要求在调用前先使用“**函数原型**”的方式声明一下：

```
3. int main(void)
4. {
5.     char name[20];
6.     printf("Input your name please!\n");
7.     gets(name);
8.     printf("Hello %s!\n",name);
9.     show(name);
10.    return 0;
11. }
```

函数调用：
函数名(参数)

```
12. void show(char str[])
13. {
14.     printf("This is the first program for %s to learn C programming!\n",str);
15. }
```

【4】 show函数定义

【3】 C语言程序的执行

一个C语言程序由若干**函数**构成，包括**main函数**、**用户自定义函数**和**库函数**。

- ◆ **程序的执行从main开始**：从main函数里面的第一条可执行语句开始，按规定的顺序依次执行里面设计好的语句；
- ◆ 当遇到对其它函数的调用时，则转向这些函数里执行（包括用户自定义函数和库函数）。在执行完函数里面的指令后退出函数，返回到之前调用它的地方继续执行。
- ◆ **main函数既是起点也是终点**，当程序执行完main函数的最后一条语句后，结束于右花括号处并返回操作系统。

【4】变量

类似数学里的变量，程序里需要定义**变量**来保存要处理的数据。

- 本例中，要保存输入的姓名，而姓名是**字符串**；
- C语言用**字符数组**保存字符串，所以定义了一个长度为20的字符数组name：

char name[20];

变量的数据类型 变量名 如果是数组，要说明数组的长度

- name被作为参数在调用gets时使用：**gets(name);**

定义其它类型的变量：

各种单一变量

```
int age;
```

```
char flag;
```

```
float score;
```

```
double dist
```

各种类型的数组

```
int x[10];
```

```
char a[7];
```

```
float f[3];
```

```
double t[8];
```

【5】关于printf函数的进一步说明

```
#include <stdio.h>
void show(char str[]);
int main(void)
{
```

```
    char name[20];
    printf("Input your name please!\n");
    gets(name);
```

```
    printf("Hello %s!\n",name);
```

```
    show(name);
    return 0;
}
```

printf函数可以只简单地将参数里双引号之间的内容输出（不含双引号）



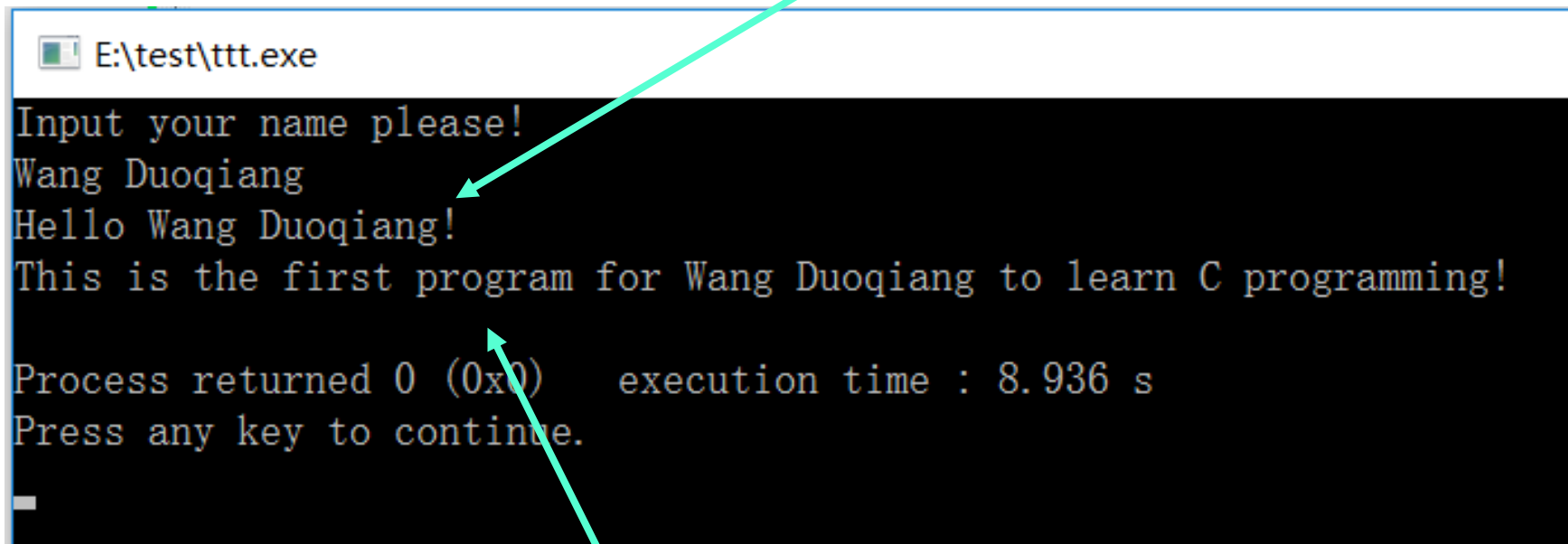
printf函数还可以实现复杂形式的输出：用后面参数的内容替换双引号中由%引导的位置，如：%s表示用name（str）里面的字符串内容替换“%s”的出现。



```
void show(char str[])
{
    printf("This is the first program for %s to learn C programming!\n",str);
}
```

例1.1的程序的結果

`printf("Hello %s!\n",name);`



```
E:\test\ttt.exe
Input your name please!
Wang Duoqiang
Hello Wang Duoqiang!
This is the first program for Wang Duoqiang to learn C programming!

Process returned 0 (0x0)   execution time : 8.936 s
Press any key to continue.
_
```

`printf("This is the first program for %s to learn C programming!\n",str);`

通过上面的例子，你学到了**两个库函数**的使用方法：

- ◆ **gets**：接收从键盘输入的以回车键结束的一行字符串
- ◆ **printf**：按照格式串要求输出信息

自学以下函数（**见第三章**）：

- ◆ **getchar**：用于接收从键盘输入的一个字符，例：`c=getchar();`
- ◆ **putchar**：将参数输出为一个字符，例：`putchar(c);`
- ◆ **puts**：输出参数数组中的字符串，例：`puts(name);`
- ◆ **scanf**：与printf相对应的格式化输入函数，例：

`scanf("%s", name);`

最后的最后：怎么成为编程高手？

何以解忧，唯有多编！



本章小结

- 首先介绍了C语言程序设计的第一个例子，说明了如何创建并运行第一个C程序，同时对第一个C程序进行了解释分析。
- 从C语言的产生与发展、C语言标准化、C语言特征等方面对C语言进行了概述。
- 简单介绍了计算机硬件系统的基本概念。
- 从进位计数制、数的机器码表示，以及字符的编码表示等三个方面介绍了数和字符的机器码表示。尤其是ASCII码，要熟练掌握。
- 在算法方面，介绍了算法的定义、表示和程序实现。
- 最后从学习并理解C语言的语法和语义、学习并掌握一些基本数据结构 and 常用算法的设计、熟悉集成开发环境，以及熟悉库函数四个方面介绍了C语言的学习方法。

本章作业

1. 就C语言的产生发展、标准化历程、语言特征等，写一篇研究报告，字数 ≥ 500 。
2. 回答问题：什么是语言类型的强弱，为什么说C语言是弱类型的？
3. 回答一下问题：
 - (1) 汉字的区位码、国标码、机内码是什么？它们之间的转换关系式什么？
 - (2) 字库是什么？点阵字库中的字模的作用是什么？
 - (3) 汉字是如何输入计算机的？计算机内存的汉字信息是什么？汉字是如何显示在屏幕上的？
4. 什么是包含命令？`#include <stdio.h>`的作用是什么？
5. 第一章习题：1.4 1.7 1.8 1.9 1.12